Lec33

31 March 2026 10:38 AM

* space complexity → heap → array/object
  → stack → function

int n = 1 → constant

int j = 0

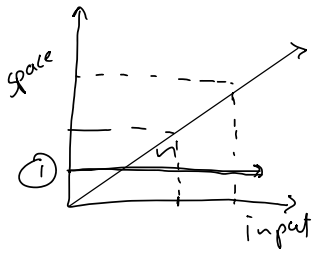
1000 ↑ arr[] = new int[n]

arr = {1, 2, 3, 4, 5} target = 4

prefix = n ✓

suffix = n

n = 1, 100, 1000

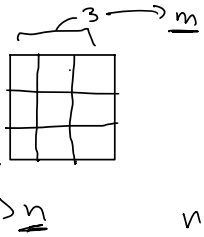


2d array

int [] arr = new int[n]

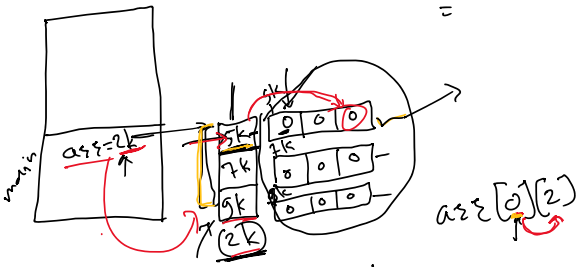
int [][] arr = new int[n][m]

row, col



n × m

=



arr[0][2]

Total number of 1d array

n+1

int arr[][] = new int[5][3]

row, col

Sout(arr) → 2k

Sout(arr[0]) → 5k

Sout(arr[0][0]) → 0 ✓

	0	1	2
0	0 (0-0)	0 (0-1)	0 (0-2)
1	0	0 (1-1)	0
2	0	0	0
3	0	0	0

arr[i]

arr =

[[I@27fe3806 → [ ][ ] arr =

[I@5f71c76a [ ] → arr[0]

0 → arr[0][2]

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

arr[0][0] = 1
arr[0][1] = 2
arr[0][2] = 3

arr[1][0] = 4
arr[1][1] = 5
arr[1][2] = 6

arr[2][0] = 7
arr[2][1] = 8
arr[2][2] = 9

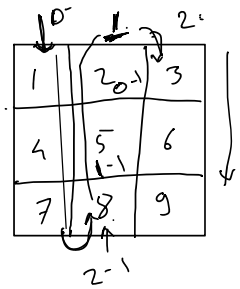
```
int row = sc.nextInt()
int col = sc.nextInt()

int arr[][] = new int[row][col]

for(i=0;i<row;i++){ ——→ n
    for(j=0;j<col;j++){ ——→ m
        arr[i][j] = sc.nextInt()
    }
}
```

n × m

n^2



1 4 7 8 5 2 3 6 9

```
row = arr.length
col = arr[0].length

for(i=0;i<col;i++){
    if(i%2==0){
        for(j=0;j<row;j++){
            Sout(arr[j][i]
```

```
→ for(j=0; j<row; j++){
        cout(arr[j][i]
    }
}else{
    for(j=row-1; j>=0; j--){
        cout(arr[j][i]
    }
}
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

→

	0	1	2	3
0	1	5	9	13
1	2	6	10	14
2	3	7	11	15
3	4	8	12	16

	0	1	2	3
0	1	5	9	13
1	2	6	10	14
2	3	7	11	15
	4	8	12	16

0 → (0-1) ⟷ (1-0), (0-3)-(3-0), (0-3) ⟷ (3-0)

1 → (1-2) ⟷ (2-1), (1-3) (3-1)

2 → (2-3) ⟷ (3-2)

3 →

```
for(i=0; i<row; i++){
    for(j=i+1; j<col; j++){
        temp = arr[i][j]
        arr[i][j] = arr[j][i]
        arr[j][i] = temp
    }
}
```